TP5 - Reverse Proxy et Load Balancing avec Traefik et Docker

Contexte : DevTech Solutions

L’équipe DevOps souhaite mettre en place un point d’entrée unique pour plusieurs applications Web. Elle veut automatiser le routage HTTP/HTTPS, découvrir les services Docker automatiquement et préparer une architecture évolutive.

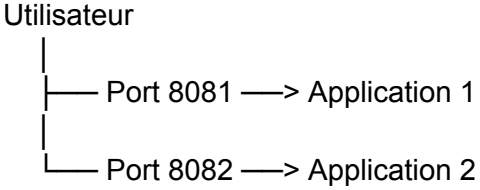
Ce TP permet de découvrir Traefik, de configurer un reverse proxy, de router deux applications Nginx et de comprendre comment Traefik peut distribuer le trafic entre plusieurs instances d’un même service.

1. Contexte entreprise

L’équipe DevTech Solutions héberge plusieurs applications dans des conteneurs Docker.

Sans reverse proxy, chaque application doit généralement être exposée par un port distinct, ou être publiée au moyen d’un autre mécanisme réseau.

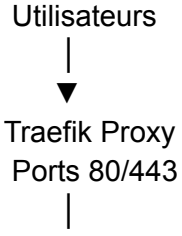
Architecture initiale

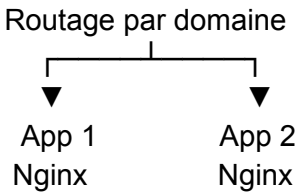


Cette architecture fonctionne, mais elle présente plusieurs limites :

- multiplication des ports exposés ;
- gestion du routage dispersée ;
- configuration manuelle lorsque de nouveaux services apparaissent ;
- gestion HTTPS plus complexe ;
- absence de point d’entrée centralisé.

Architecture cible avec Traefik





Objectif : centraliser l'entrée du trafic et automatiser le routage vers les services Docker.

Bonnes pratiques de sécurité

Identifier les applications publiques, les applications internes et les interfaces d'administration. Autoriser uniquement les flux nécessaires entre ces zones. Le reverse proxy centralise les accès, mais sa compromission ou son indisponibilité peut affecter plusieurs applications.

2. User Story

US-DEVOPS-001 — Mettre en place un reverse proxy avec Traefik

En tant qu'ingénieur DevOps, je veux déployer Traefik avec Docker et configurer le routage dynamique vers plusieurs applications, afin de centraliser l'accès aux services et de faciliter leur évolution.

Critères d'acceptation de sécurité supplémentaires

- Aucun port applicatif Nginx n'est publié sur l'hôte.
- Un conteneur sans `traefik.enable=true` ne reçoit aucune route Docker.
- Le dashboard et l'API sont inaccessibles depuis une machine non autorisée.
- Les secrets et les clés privées sont absents des labels Docker et du dépôt Git.
- Pour la variante HTTPS, le certificat est vérifié sans utiliser `curl -k`.

Critères d'acceptation :

- Traefik démarre correctement.
- Le dashboard est accessible dans l'environnement de laboratoire.
- Deux applications Nginx sont découvertes automatiquement.
- Chaque application est accessible par son nom d'hôte.
- Les services ne sont pas exposés automatiquement sans autorisation explicite.
- Les routes et les services sont visibles dans le dashboard.
- Une troisième instance peut être ajoutée pour expérimenter le Load Balancing.

3. Objectifs pédagogiques

À la fin de ce TP, l'étudiant devra être capable de :

- expliquer le rôle d'un reverse proxy ;

- distinguer un reverse proxy d'un forward proxy ;
- expliquer le principe du Load Balancing ;
- comprendre le rôle de Traefik ;
- comprendre les providers et la découverte automatique ;
- distinguer la configuration statique de la configuration dynamique ;
- configurer les entrypoints HTTP et HTTPS ;
- utiliser Docker Compose ;
- configurer les routers, services et middlewares ;
- utiliser les labels Docker pour définir des routes ;
- tester le routage avec `curl` ;
- vérifier les routes à travers le dashboard et l'API ;
- comprendre la différence entre TLS activé et certificat valide ;
- tester la répartition de charge entre plusieurs instances ;
- identifier les risques liés à l'exposition du dashboard et du socket Docker.

4. Prérequis

L'étudiant doit disposer de :

- Docker Engine ou Docker Desktop ;
- Docker Compose v2 ;
- un terminal Linux, macOS ou Windows avec WSL ;
- un navigateur Web ;
- les commandes `curl` et `docker` ;
- les ports 80, 443 et 8080 disponibles pour ce laboratoire.

Vérifier les installations :

```
docker --version
docker compose version
curl --version
```

Documentation officielle :

- Documentation Traefik
- Documentation Docker Compose
- Provider Docker de Traefik

5. Rappel — Qu'est-ce qu'un reverse proxy ?

Un reverse proxy reçoit les requêtes des clients et les transmet aux serveurs applicatifs appropriés.

Sans reverse proxy :

Client ——————————> Serveur Web

Avec un reverse proxy :

Client
  |
  ▼
Reverse Proxy
  |
  ▼
Serveur Web

Le client communique avec le reverse proxy, qui prend en charge la sélection de la destination.

Un reverse proxy peut notamment :

- centraliser l'accès aux applications ;
- router les requêtes selon le nom d'hôte ou le chemin URL ;
- terminer les connexions TLS ;
- appliquer des middlewares ;
- répartir le trafic entre plusieurs serveurs ;
- faciliter l'observabilité et la gestion du trafic.

Exemple :

- `app1.localhost` dirige vers l'application 1 ;
- `app2.localhost` dirige vers l'application 2.

L'utilisateur n'a pas besoin de connaître les adresses internes des conteneurs.

Principe essentiel : le reverse proxy constitue un point d'entrée logique entre les clients et les applications.

Ne pas confondre routage et autorisation

Une règle `Host(...)` sélectionne une application ; elle ne constitue pas un contrôle d'accès. Un client peut choisir son en-tête `Host`.

Une application privée doit disposer d'une authentification et d'autorisations adaptées. Le reverse proxy ne remplace pas la sécurisation du code applicatif.

6. Traefik : présentation et concepts essentiels

Traefik est un reverse proxy et un load balancer conçu pour les environnements modernes et dynamiques.

Il peut découvrir les services provenant de plusieurs sources, notamment :

- Docker ;
- Kubernetes ;
- des fichiers de configuration ;
- d'autres providers pris en charge.

Dans notre TP, le provider Docker permet à Traefik de détecter les conteneurs autorisés et de construire les routes correspondantes à partir de leurs labels.

Les composants principaux

Composant	Rôle
EntryPoint	Définit le point d'entrée réseau, par exemple `:80` ou `:443`.
Router	Détermine quelle règle doit recevoir une requête.
Service	Définit la destination applicative et les serveurs backend.
Middleware	Modifie ou contrôle le traitement d'une requête.
Provider	Fournit à Traefik les informations sur les routes et les services.
Dashboard	Permet d'observer les routers, services et middlewares configurés.

Exemple de traitement :

Requête HTTP



Principe essentiel : Traefik sépare le point d’entrée, la décision de routage et la destination applicative.

Réduire la surface d’exposition

Activer uniquement les entrypoints, providers et interfaces nécessaires. Documenter, pour chaque router, son domaine, son service cible et ses middlewares de sécurité.

Ne pas considérer un nom de domaine difficile à deviner comme une protection.

7. Configuration statique et dynamique

Traefik distingue deux types de configuration.

Configuration statique

Elle définit les éléments nécessaires au démarrage de Traefik :

- les entrypoints ;
- les providers ;
- les paramètres du dashboard ;
- les paramètres de journalisation ;
- les options générales du proxy.

Configuration dynamique

Elle définit les éléments qui déterminent comment les requêtes sont traitées :

- les routers ;
- les services ;
- les middlewares ;
- les règles de routage ;
- les paramètres TLS associés aux routes.

Dans notre TP, la configuration statique est stockée dans `config/traefik.yaml`. La configuration dynamique est principalement découverte depuis les labels des conteneurs Docker.

Principe essentiel : la configuration statique initialise Traefik ; la configuration dynamique décrit les routes et services qu'il doit gérer.

Protéger les changements

Limiter les droits de modification des fichiers de configuration et des définitions Compose. Une modification de labels peut exposer une application ou détourner son trafic.

Versionner les configurations sans secrets, faire relire les changements sensibles et conserver une version permettant un retour arrière.

8. Structure du projet

Créer l'arborescence suivante :

```
traefik-tp/
├── docker-compose.yml
├── config/
│   └── traefik.yaml
├── app1/
│   └── index.html
└── app2/
    └── index.html
```

Commandes :

```
mkdir -p traefik-tp/config
mkdir -p traefik-tp/app1
mkdir -p traefik-tp/app2

cd traefik-tp
```

Créer les pages Web :

```
echo '<h1>App1 - Bonjour depuis App1</h1>' \
  > app1/index.html

echo '<h1>App2 - Bonjour depuis App2</h1>' \
  > app2/index.html
```

Résultat attendu : deux pages HTML distinctes sont disponibles pour tester le routage.

Placer uniquement les fichiers destinés à être servis dans `app1/`, `app2/` et `app1b/`. Ne jamais y déposer de fichiers `.env`, de clés privées, de sauvegardes ou de fichiers d'authentification.

Conserver les secrets hors des répertoires Web et du dépôt Git, avec des permissions restrictives. Un fichier `.gitignore` ne protège pas un secret déjà publié.

9. Phase 1 — Déployer Traefik

9.1 Créer le réseau Docker

Nous allons utiliser un réseau Docker partagé nommé `frontend`.

docker network create frontend

Si le réseau existe déjà, Docker peut signaler qu'il est présent. Dans ce cas, on peut continuer.

Vérifier : docker network ls

Pourquoi créer un réseau partagé ?

- Traefik doit pouvoir communiquer avec les conteneurs applicatifs.
- Les applications peuvent rester accessibles uniquement sur le réseau Docker.
- Il n'est pas nécessaire de publier un port hôte pour chaque application.

9.2 Créer la configuration statique

Créer le fichier `config/traefik.yaml` :

https://github.com/traefik/traefik/blob/master/traefik.sample.yml

```
global:
  checkNewVersion: false
  sendAnonymousUsage: false

log:
  level: DEBUG

api:
  dashboard: true
  insecure: true
```

```
entryPoints:
  web:
    address: ":80"

  websecure:
    address: ":443"
```

- `log.level: DEBUG` active des journaux détaillés pour le laboratoire.
- `api.dashboard: true` active l'API et le dashboard.
- `api.insecure: true` expose le dashboard sur le port dédié non sécurisé par défaut.
- `web` écoute sur le port 80.
- `websecure` écoute sur le port 443.

Attention : `api.insecure: true` est acceptable uniquement dans un laboratoire isolé. Ne pas exposer cette configuration telle quelle sur Internet ou en production.

9.3 Créer `docker-compose.yml`

```
services:
  traefik:
    image: traefik:v3.5.2
    container_name: traefik-1

    ports:
      - "80:80"
      - "443:443"
      - "127.0.0.1:8080:8080"

    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - ./config/traefik.yaml:/etc/traefik/traefik.yaml:ro

    command:
      - "--configFile=/etc/traefik/traefik.yaml"

    networks:
      - frontend

    restart: unless-stopped

networks:
  frontend:
    external: true
```

Le port 8080 est lié à `127.0.0.1` pour limiter l'accès au dashboard à la machine locale.

Le montage du socket Docker reste sensible, même en lecture seule : un accès à l'API Docker peut présenter des risques importants. En environnement de production, il faut étudier des mécanismes d'accès plus restrictifs.

9.4 Démarrer Traefik

docker compose up -d

Vérifier le conteneur :

docker ps

Consulter les journaux :

docker logs traefik-1

Accéder au dashboard :

Ouvrir le dashboard Traefik

À cette étape, il est possible que les routes applicatives ne soient pas encore visibles : nous n'avons pas encore activé le provider Docker.

Résultat attendu : Traefik est démarré et ses entrypoints sont initialisés.

Protéger le proxy et l'hôte Docker

Conserver `127.0.0.1:8080:8080` pour le laboratoire. En production, désactiver `api.insecure`, supprimer cette publication et réserver l'administration à un réseau privé ou à un VPN.

Le montage `docker.sock:ro` **ne rend pas l'API Docker accessible uniquement en lecture**. Prévoir un accès filtré aux opérations nécessaires, par exemple au moyen d'un proxy de socket correctement configuré, isolé et non publié. Ne jamais monter ce socket dans les applications.

Réserver `DEBUG` au diagnostic temporaire. Éviter `privileged: true` et ajouter `no-new-privileges` aux conteneurs compatibles, après vérification de leur fonctionnement.

10. Phase 2 — Activer la découverte automatique Docker

10.1 Modifier `config/traefik.yaml`

https://doc.traefik.io/traefik/reference/routing-configuration/dynamic-configuration-methods/

https://community.traefik.io/t/simple-example-with-configuration-files-static-and-dynamic/26981

```
global:
  checkNewVersion: false
  sendAnonymousUsage: false

log:
  level: DEBUG

api:
  dashboard: true
  insecure: true

entryPoints:
  web:
    address: ":80"

  websecure:
    address: ":443"

providers:
  docker:
    endpoint: "unix:///var/run/docker.sock"
    exposedByDefault: false
    network: frontend
```

Le paramètre :

exposedByDefault: false

indique que Traefik ne doit pas exposer automatiquement tous les conteneurs découverts. Les conteneurs doivent explicitement porter le label `traefik.enable=true`.

Le paramètre `network: frontend` indique le réseau que Traefik doit utiliser pour joindre les services, ce qui évite certaines ambiguïtés lorsque les conteneurs sont attachés à plusieurs réseaux.

Redémarrer Traefik : docker compose up -d

Vérifier : docker logs traefik-1

Principe essentiel : la découverte automatique simplifie la configuration, mais l'exposition des services doit rester explicite.

Garder une exposition explicite

Conserver `exposedByDefault: false` et autoriser individuellement les applications. Cette mesure limite l'exposition accidentelle, mais une personne capable de créer des conteneurs et de modifier leurs labels peut aussi demander leur exposition.

Limiter donc les droits d'accès à Docker et aux pipelines de déploiement. Ne pas placer de mots de passe ou de jetons dans les labels.

Point de vigilance : après modification de la configuration statique montée dans le conteneur, redémarrer explicitement Traefik avec `docker compose restart traefik`. Un simple `docker compose up -d` ne garantit pas son redémarrage si Compose ne détecte aucun changement de définition.

11. Phase 3 — Configurer les deux applications Nginx

11.1 Ajouter les services dans Docker Compose

Remplacer `docker-compose.yml` par la version complète suivante :

```
services:
  traefik:
    image: traefik:v3.5.2
    container_name: traefik-1

    ports:
      - "80:80"
      - "443:443"
      - "127.0.0.1:8080:8080"

    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
```

```
    - ./config/traefik.yaml:/etc/traefik/traefik.yaml:ro

  command:
    - "--configFile=/etc/traefik/traefik.yaml"

  networks:
    - frontend

  restart: unless-stopped

app1:
  image: nginx:stable
  container_name: nginx-app1

  volumes:
    - ./app1:/usr/share/nginx/html:ro

  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.app1.rule=Host(`app1.localhost`)"
    - "traefik.http.routers.app1.entrypoints=web"
    - "traefik.http.services.app1.loadbalancer.server.port=80"

  networks:
    - frontend

  restart: unless-stopped

app2:
  image: nginx:stable
  container_name: nginx-app2

  volumes:
    - ./app2:/usr/share/nginx/html:ro

  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.app2.rule=Host(`app2.localhost`)"
    - "traefik.http.routers.app2.entrypoints=web"
    - "traefik.http.services.app2.loadbalancer.server.port=80"

  networks:
    - frontend

  restart: unless-stopped

networks:
```

```
frontend:
  external: true
```

Dans cette version, nous configurons uniquement le routage HTTP afin de valider le fonctionnement de base avant d'ajouter TLS.

11.2 Comprendre les labels

Label	Fonction
`traefik.enable=true`	Autorise l'exposition du conteneur par Traefik.
`routers.app1.rule=Host(...)`	Définit la règle correspondant au nom d'hôte.
`routers.app1.entrypoints=web`	Associe le routeur à l'entrée HTTP.
`services.app1.loadbalancer.server.port=80`	Définit le port interne du serveur Nginx.

Le port `80` correspond au port du conteneur, pas au port publié sur l'hôte.

Point important : Traefik utilise ici un service logique pour chaque application. Un service logique peut ensuite contenir plusieurs instances backend, ce qui nous permettra d'expérimenter le Load Balancing.

11.3 Démarrer les applications

docker compose up -d

Vérifier :

docker ps

Résultat attendu :

traefik-1
nginx-app1

nginx-app2

Appliquer le moindre privilège

Ne pas ajouter de section `ports` aux applications. Conserver les montages de contenu en lecture seule.

Utiliser des images de confiance, identifier précisément les versions déployées et organiser leurs mises à jour. Lorsque l'image le permet, exécuter Nginx avec un utilisateur non privilégié, réduire les capacités Linux et rendre le système de fichiers en lecture seule en prévoyant ses répertoires temporaires nécessaires.

Le réseau `frontend` partagé facilite le TP, mais n'isole pas les applications entre elles. Pour des applications de niveaux de confiance différents, prévoir des réseaux séparés auxquels Traefik est rattaché.

12. Phase 4 — Tester le routage HTTP

12.1 Accéder aux applications

Ouvrir :

- http://app1.localhost
- http://app2.localhost

Résultat attendu :

app1.localhost
→ Bonjour depuis App1

app2.localhost
→ Bonjour depuis App2

Les deux applications utilisent le même point d'entrée HTTP, sur le port 80. Traefik les différencie grâce au nom d'hôte.

12.2 Tester avec curl

curl http://app1.localhost
curl http://app2.localhost

On peut également tester la règle de routage en utilisant explicitement l'en-tête `Host` :

curl -H "Host: app1.localhost" http://localhost
curl -H "Host: app2.localhost" http://localhost

Question : pourquoi les deux requêtes peuvent-elles utiliser le même port tout en arrivant dans des applications différentes ?

Réponse attendue : parce que Traefik sélectionne le router en fonction de la règle `Host`.

Tester également les refus

Utiliser uniquement des données fictives tant que les échanges restent en HTTP.

En complément des tests fonctionnels :

- vérifier qu'un nom d'hôte inconnu n'atteint aucune application ;
- vérifier l'absence de ports Nginx publiés sur l'hôte ;
- vérifier qu'un conteneur sans autorisation explicite n'apparaît pas comme application routée ;
- effectuer un test depuis une machine non autorisée pour contrôler les restrictions réseau.

Une réponse `404` à une requête ne prouve pas, à elle seule, que toute l'architecture est sécurisée.

13. Phase 5 — Vérifier le dashboard et l'API

Ouvrir :

Dashboard Traefik

Vérifier les éléments suivants :

- les routers `app1` et `app2` ;
- les services associés ;
- les entrypoints utilisés ;
- l'état de configuration des routes.

Interroger l'API :

curl http://localhost:8080/api/http/routers

Puis :

curl http://localhost:8080/api/http/services

Ces endpoints retournent des informations JSON permettant d'inspecter les routes et services connus de Traefik.

Question : quelle différence observes-tu entre un router et un service ?

Réponse attendue : le router décide où une requête doit aller ; le service représente la destination vers laquelle Traefik transmet cette requête.

14. Phase 6 — Introduire le Load Balancing

Jusqu'ici, chaque application ne possède qu'une seule instance Nginx. Traefik réalise du reverse proxy et du routage, mais il n'y a pas encore de répartition entre plusieurs instances pour une même application.

Nous allons ajouter une deuxième instance de l'application 1.

14.1 Créer une deuxième instance Nginx

Ajouter dans `docker-compose.yml` :

```
 app1b:
    image: nginx:stable
    container_name: nginx-app1b

    volumes:
      - ./app1b:/usr/share/nginx/html:ro

    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.app1.rule=Host(`app1.localhost`)"
      - "traefik.http.routers.app1.entrypoints=web"
      - "traefik.http.services.app1.loadbalancer.server.port=80"

    networks:
      - frontend

    restart: unless-stopped
```

Avant de démarrer le conteneur, créer son répertoire et une page distincte :

```
mkdir -p app1b

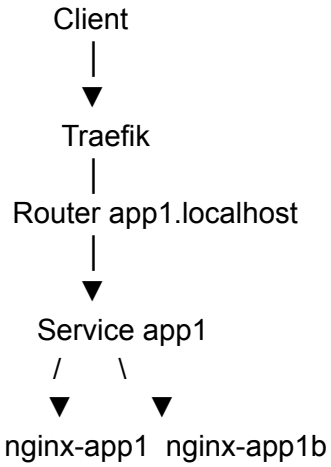
echo '<h1>Bonjour depuis App1 - Instance B</h1>' \
  > app1b/index.html
```

Attention : pour cet exercice, les labels `router` et `service` d'`app1b` doivent reprendre exactement les mêmes noms que ceux de l'application 1. Cela permet à Traefik de regrouper les deux conteneurs sous un service logique commun. Ne pas définir une seconde règle de router contradictoire.

Démarrer : docker compose up -d

Traefik devrait maintenant détecter deux instances du service `app1`.

14.2 Comprendre la répartition



Traefik peut répartir les requêtes entre les deux backends. La stratégie exacte et le résultat de chaque requête dépendent du mécanisme de Load Balancing et de l'état des instances.

Pour constater la répartition, les deux instances doivent servir des réponses identifiables. On peut vérifier dans le dashboard que les deux backends apparaissent dans le service `app1`, puis multiplier les requêtes :

```
for i in $(seq 1 10); do
  curl -s http://app1.localhost
  echo
done
```

Si les réponses sont identiques ou si une seule instance apparaît, vérifier les labels, le réseau Docker, les journaux Traefik et la configuration du service.

Principe essentiel : le routage sélectionne l'application ; le Load Balancing sélectionne une instance backend parmi celles qui appartiennent à cette application.

15. Phase 7 — Comprendre HTTPS et TLS

Traefik écoute sur le port 443 grâce à l'entrypoint `websecure`. Cependant, écouter sur ce port ne suffit pas pour fournir un certificat TLS valide.

Pour préparer une route HTTPS, on peut ajouter les labels suivants à un service :

```
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.app1-secure.rule=Host(`app1.localhost`)"
  - "traefik.http.routers.app1-secure.entrypoints=websecure"
  - "traefik.http.routers.app1-secure.tls=true"
  - "traefik.http.services.app1.loadbalancer.server.port=80"
```

Cela active le traitement TLS pour le router, mais ne crée pas à lui seul un certificat publiquement reconnu.

Pour un laboratoire local, on peut utiliser un certificat de développement ou une autorité de certification locale. Pour une application publique, on peut configurer un resolver ACME, par exemple avec Let's Encrypt, sous réserve de respecter ses exigences de validation de domaine et d'accessibilité.

Attention : ne pas confondre activation de TLS et possession d'un certificat valide. Le test `curl -k` ignore les erreurs de validation du certificat et ne constitue pas une validation de sécurité.

Vérifier la confiance, pas seulement le chiffrement

Utiliser un certificat correspondant au nom demandé et reconnu par le client : autorité locale approuvée pour le laboratoire, autorité reconnue pour un domaine public.

Protéger les clés privées et les données ACME, surveiller l'expiration des certificats et tester le renouvellement. Réserver `curl -k` au diagnostic.

Prévoir une politique TLS moderne, avec TLS 1.2 au minimum. Activer la redirection HTTP vers HTTPS après validation du fonctionnement HTTPS. Déployer HSTS uniquement lorsque les domaines concernés fonctionnent durablement en HTTPS.

La terminaison TLS sur Traefik ne chiffre pas automatiquement le flux Traefik–Nginx : évaluer aussi la protection du réseau backend.

16. Phase 8 — Comprendre les middlewares

Un middleware permet d'appliquer un traitement supplémentaire avant de transmettre une requête au service backend.

Exemples d’utilisation :

- redirection HTTP vers HTTPS ;
- authentification ;
- limitation de débit ;
- ajout ou suppression d’en-têtes ;
- réécriture de chemins.

Exemple de middleware de redirection vers HTTPS :

```
http:
  middlewares:
    redirect-to-https:
      redirectScheme:
        scheme: https
        permanent: true
```

Cette configuration est un exemple de configuration dynamique au format fichier. Elle doit être fournie à Traefik via un provider de fichiers activé dans la configuration statique, ou être traduite en labels Docker adaptés.

Question : pourquoi un middleware peut-il être utile avant même que la requête atteigne le backend ?

Réponse attendue : parce qu’il permet d’appliquer des règles transversales sans les réimplémenter dans chaque application.

Appliquer et vérifier les protections

Associer explicitement les middlewares aux routers concernés : leur définition seule ne suffit pas.

Utiliser selon le besoin une authentification, une liste d’adresses autorisées, une limitation de débit et des en-têtes de sécurité adaptés. Utiliser BasicAuth uniquement via HTTPS et conserver ses données d’authentification dans un fichier protégé.

Si Traefik se trouve derrière un autre proxy, ne faire confiance aux en-têtes d’adresse client que pour les proxies identifiés. Tester les restrictions depuis un client réellement extérieur au périmètre autorisé.

17. Phase 9 — Dépannage

Incident 1 — Traefik ne démarre pas

Vérifier :

docker logs traefik-1
docker ps -a

Causes possibles :

- fichier YAML invalide ;
- port 80, 443 ou 8080 déjà utilisé ;
- réseau Docker manquant ;
- fichier de configuration mal monté.

Incident 2 — Le router n’apparaît pas

Vérifier :

docker inspect nginx-app1
docker logs traefik-1

Causes possibles :

- label `traefik.enable=true` absent ;
- provider Docker non activé ;
- réseau incorrect ;
- règle de routage mal formée.

Incident 3 — Erreur 404

Une réponse 404 peut indiquer qu’aucun router ne correspond à la requête.

Vérifier :

- le nom d’hôte ;
- la règle `Host(...)` ;
- l’entrypoint ;
- la présence du router dans le dashboard.

Incident 4 — Erreur 502

Une erreur 502 peut indiquer que Traefik ne parvient pas à communiquer correctement avec le backend.

Vérifier :

- que le conteneur Nginx est démarré ;
- que le port interne est correct ;
- que Traefik et Nginx partagent le bon réseau ;
- que les journaux du backend ne révèlent pas d’erreur.

Incident 5 — Dashboard vide

Vérifier :

- l'activation du provider Docker ;
- le montage du socket Docker ;
- les labels des conteneurs ;
- les journaux Traefik.

18. Travail demandé — Exercices pratiques

Exercice 1 — Déploiement initial

1. Créer le réseau `frontend`.
2. Déployer Traefik.
3. Accéder au dashboard.
4. Vérifier les ports exposés.
5. Expliquer la différence entre un entrypoint et un router.

Exercice 2 — Routage par nom d'hôte

1. Déployer `app1` et `app2`.
2. Configurer leurs labels.
3. Tester `app1.localhost`.
4. Tester `app2.localhost`.
5. Identifier les routers et services dans le dashboard.

Exercice 3 — Load Balancing

1. Ajouter une deuxième instance de l'application 1.
2. Donner une réponse distincte à chaque instance.
3. Vérifier les deux backends dans le dashboard.
4. Envoyer plusieurs requêtes HTTP.
5. Expliquer pourquoi le Load Balancing est différent du routage par nom d'hôte.

Exercice 4 — Sécurité

1. Expliquer le risque associé à `api.insecure: true`.
2. Expliquer pourquoi le socket Docker est sensible.
3. Proposer une approche pour protéger le dashboard.

4. Expliquer pourquoi `traefik.enable=true` est préférable à l'exposition automatique de tous les conteneurs.
5. Distinguer TLS activé et certificat valide.

Exercice 5 — Évolution de l'architecture

Proposer une architecture Traefik pour les besoins suivants :

- trois applications différentes ;
- deux instances backend pour l'application principale ;
- une route HTTPS ;
- un middleware d'authentification ;
- une politique de santé et de disponibilité ;
- un dashboard non accessible publiquement.

Présenter le schéma d'architecture et les labels ou configurations nécessaires.

19. Questions de compréhension

1. Qu'est-ce qu'un reverse proxy ?
2. Quelle est la différence entre un reverse proxy et un Load Balancer ?
3. Quel est le rôle de Traefik dans une architecture Docker ?
4. Quelle est la différence entre la configuration statique et dynamique ?
5. À quoi sert un provider ?
6. Quel est le rôle d'un entrypoint ?
7. Quelle est la différence entre un router et un service ?
8. Pourquoi utilise-t-on des labels Docker ?
9. Quel est l'intérêt de `exposedByDefault: false` ?
10. Comment Traefik peut-il découvrir automatiquement un nouveau conteneur ?
11. Quelle est la différence entre routage et Load Balancing ?
12. Pourquoi faut-il surveiller la santé des backends ?
13. Pourquoi le dashboard doit-il être protégé ?
14. Pourquoi un router TLS ne garantit-il pas à lui seul un certificat valide ?
15. Comment ajouter une nouvelle instance backend sans changer le point d'entrée client ?
16. Pourquoi une règle `Host(...)` ne constitue-t-elle pas une authentification ?
17. Pourquoi `docker.sock:ro` ne garantit-il pas une API Docker en lecture seule ?
18. Pourquoi faut-il protéger l'API autant que le dashboard ?
19. Quelle différence existe entre un réseau partagé et une segmentation réseau ?
20. Pourquoi deux instances sur le même hôte ne suffisent-elles pas à assurer une haute disponibilité ?
21. Quelles preuves permettent de vérifier qu'un contrôle de sécurité fonctionne ?

20. Construire une cartographie exploitable pour la sécurité

La cartographie doit relier les composants, les flux, les risques et les vérifications. Les protections ci-dessous constituent une **cible à vérifier**, et non des mesures considérées comme déjà appliquées.

Priorités : **P1** = à traiter avant toute exposition hors du laboratoire ; **P2** = durcissement et exploitation. Les responsables proposés sont à confirmer par l'équipe.

Composant ou flux	Exposition et données	Risque principal	Protection attendue	Vérification ou preuve	Responsable proposé	Priorité
Clients → Traefik, ports 80/443	Entrée des applications ; requêtes et éventuels identifiants	Interception, accès non autorisé, saturation	HTTPS valide, exposition réseau limitée au besoin, authentification des applications privées	`curl` sans `-k`, test de redirection et test des restrictions	DevOps / Réseau	P1
Administrateurs → dashboard et API	Configuration des routes et adresses internes	Divulgation d'informations techniques	Accès local dans le TP ; réseau privé ou VPN et authentification en production	Test local autorisé et test distant refusé ; contrôle de `/api` et `/dashboard/`	DevOps	P1

Traefik → API Docker	Socket Unix ou accès filtré ; métadonnées Docker	Compromission de l'hôte	Accès minimal filtré, aucun accès depuis les applications, aucune publication publique	Inspection des montages et réseaux ; vérification des opérations autorisées	Administrateur système	P1
Traefik → App1 et App1B	Réseau backend ; trafic applicatif	Contournement du proxy, indisponibilité	Aucun port hôte publié, accès réseau maîtrisé, contrôles de santé	Inspection Compose et test d'arrêt d'une instance	DevOps / Équipe App1	P1
Traefik → App2, puis App3	Réseaux applicatifs ; trafic propre à chaque application	Exposition involontaire, déplacement latéral	Labels explicites, séparation réseau selon le niveau de confiance	Inventaire des routes et tests de connectivité entre réseaux	DevOps / Équipes applicatives	P1
Configurations et labels	Fichiers Compose, YAML, règles de routage	Modification non autorisée, exposition accidentelle	Droits restreints, revue des changements, versionnement sans secrets	Revue des permissions et de l'historique des changements	DevOps	P1

Certificats et secrets	Clés privées, stockage ACME, fichiers d'authentification	Vol de clés, usurpation, expiration	Stockage protégé, accès minimal, renouvellement et alertes	Contrôle des permissions, de l'expiration et du renouvellement	DevOps / Sécurité	P1
Images et conteneurs	Traefik et Nginx exécutés sur l'hôte	Vulnérabilités, privilèges excessifs	Versions identifiées et corrigées, analyse des images, privilèges réduits	Inventaire des versions et rapport de vulnérabilités	DevOps	P1
Journaux et supervision	Requêtes, erreurs, informations clients	Fuite de données, absence de détection, disque saturé	Accès restreint, filtrage des données sensibles, rotation et alertes	Inspection d'un échantillon et test d'alerte	Exploitation / Sécurité	P2
Hôte Docker et instance Traefik	Infrastructure commune à toutes les applications	Panne générale, perte de configuration	Sauvegarde, restauration testée ; redondance selon les objectifs de disponibilité	Compte rendu de restauration et test de panne	Infrastructure	